

ÉVALUATION FINALE

BookList Pro

Le cahier de lecture des Comptoirs du Livre

Vous êtes l'équipe mobile. À vous de livrer l'application, puis de la défendre en comité de recette.

Organisation **Équipes de 2 à 3 personnes**

Exécution **Navigateur — aucun émulateur requis**

API **Fournie dans api-books-v2 (voir README)**

Recette **5 minutes : démonstration, revue de code, questions**

Le contexte

Les Comptoirs du Livre est un réseau de dix-sept librairies indépendantes réparties sur trois régions. Chaque boutique tient depuis toujours un cahier de lecture : les libraires y notent les ouvrages qu'ils ont lus, leur avis, ce qu'ils recommandent à quel type de client. Ce cahier est l'outil de vente le plus précieux du réseau, et il est aujourd'hui en papier, illisible, et perdu dès qu'un libraire change de boutique.

La direction a décidé de le numériser. L'application s'appellera **BookList Pro**.

Une équipe back-end a déjà livré l'API : elle est en production, documentée, versionnée, et elle ne changera pas pour vous. **Vous êtes l'équipe mobile**. Votre mission est de livrer l'application cliente en trois jours, et de la présenter en comité de recette.

Vos utilisateurs

Profil	Ce qu'ils font	Ce que ça implique
Libraire titulaire	Ajoute, modifie et note les ouvrages, rédige les notes de lecture	Droits d'écriture complets
Libraire saisonnier	Consulte le fonds et les notes de lecture pour conseiller un client	Lecture seule, jamais d'écriture
Responsable réseau	Consulte les statistiques de lecture de l'ensemble du réseau	Tableau de bord chiffré

Les contraintes du terrain

Ces contraintes ne sont pas décoratives : chacune se traduit par une exigence technique du sujet.

- **Les boutiques sont mal connectées.** ADSL fatigué, wifi partagé avec la caisse, réserves en sous-sol sans réseau. L'API répond parfois en deux secondes, et parfois pas du tout.
- **Les libraires travaillent hors ligne.** L'inventaire se fait en réserve, les salons du livre se tiennent dans des halls sans couverture. L'application doit rester utilisable sans réseau et se resynchroniser ensuite.
- **Plusieurs personnes modifient les mêmes fiches.** Deux libraires peuvent noter le même ouvrage le même jour, l'un depuis la réserve hors ligne, l'autre depuis la caisse. Vos données entreront en conflit. C'est certain, pas hypothétique.
- **Les postes sont hétérogènes.** La direction a tranché : **la première cible est le navigateur**, sur les postes de caisse existants. Les versions iOS et Android suivront, sur la même base de code. Vous développez donc en React Native exécuté via `npx expo start --web`, sans émulateur.
- **La saisie d'un libraire ne se perd jamais.** C'est la règle numéro un du cahier des charges. Un libraire qui perd trois notes de lecture rédigées pendant l'inventaire n'utilisera plus jamais l'application.

Tout ce qui n'est pas précisé dans ce document est laissé à votre appréciation — mais chaque choix devra être justifié en comité de recette. N'hésitez jamais à demander des précisions à votre formateur, qui joue le rôle du responsable produit.

1. Votre mission

Livrer une application React Native fonctionnelle, robuste et documentée, en respectant le contrat de qualité du chapitre 3 et en atteignant le plus haut lot de livraison possible (chapitre 5).

Durée	3 jours
Organisation	Équipes de 2 à 3 personnes, ou seul avec périmètre réduit
Environnement d'exécution	Navigateur, via <code>npm expo start --web</code> — aucun émulateur requis
Cible secondaire, en bonus	Expo Go sur un appareil physique
Livraison	Dépôt Git, README, ADR, fiche IA, vidéo de démonstration
Recette	5 minutes : démonstration du scénario, revue de code, questions

Deux points d'attention sur la cible navigateur

Fonctionnalité	Adaptation attendue
Envoi d'une image de couverture	La sélection de fichier doit fonctionner sur navigateur et dégrader proprement ailleurs. L'API reçoit l'image encodée en base64 : à vous de lire le fichier et de le convertir.
Détection de la connectivité	<code>navigator.onLine</code> et les événements <code>online / offline</code> sur navigateur, <code>netinfo</code> sur mobile, derrière une seule abstraction <code>services/reseau.ts</code> .

Toute capacité indisponible sur le navigateur doit être encapsulée derrière une interface unique, avec une implémentation par plateforme. C'est un point regardé en revue de code, pas une contrainte subie.

2. L'environnement technique fourni

L'API vous est livrée dans le dossier `api-books-v2/`. Elle est complète et documentée : son `README.md` fait foi. Vous n'avez normalement rien à y modifier — signalez tout dysfonctionnement à votre formateur.

Quatre modes de lancement

Commande	Effet	Quand l'utiliser
<code>npm start</code>	Fonctionnement normal, sans authentification	Lots 1 à 3
<code>npm run auth</code>	Authentification et rôles obligatoires	Lot 4
<code>npm run chaos</code>	Latence de 1,5 s et 30 % d'échecs	Lot 4, et le plus tôt possible
<code>npm run final</code>	Authentification et mode dégradé combinés	Conditions de la recette finale

Ce que l'API impose

Pagination et filtrage côté serveur. Le fonds compte 500 ouvrages. **GET /books** renvoie un objet paginé :

```
GET /books?page=1&limit=20&q=tolkien&status=lu&favori=true&sort=titre&order=asc
→ { items, page, limit, total, totalPages }
```

Télécharger les 500 livres pour filtrer côté client est un contresens technique et sera pénalisé.

Vernement des fiches. Chaque ouvrage porte **updatedAt** et **version**. **PUT /books/:id** accepte un en-tête **If-Match** contenant la version que vous croyez à jour :

- version à jour : écriture acceptée, **version** incrémentée ;
- version périmée : réponse **409 Conflict**, avec la fiche serveur dans le corps ;
- en-tête absent : écriture acceptée sans contrôle, le dernier écrivain gagne. Toléré jusqu'au lot 3, inacceptable au lot 4.

PUT exige une représentation complète de l'ouvrage ; **PATCH** accepte une modification partielle.

Synchronisation par lot. **POST /sync** accepte un tableau de mutations, les traite dans l'ordre, et renvoie un statut par mutation : **ok**, **conflict** ou **erreur**. Une mutation dont l'identifiant a déjà été traité renvoie son résultat mémorisé sans être réappliquée : c'est le mécanisme d'idempotence. C'est à vous de générer ces identifiants côté client et de les conserver entre deux tentatives.

Statistiques. **GET /stats** fournit les totaux, la moyenne des notes, leur distribution, et la répartition par année et par auteur.

Mode dégradé. Latence artificielle et erreurs 503 aléatoires, activables par variable d'environnement. **La recette finale se déroulera dans ce mode.**

Authentification. Jeton d'accès valable 120 secondes, jeton de rafraîchissement, deux rôles : **lecteur** en lecture seule, **editeur** en écriture. Désactivée par défaut, elle n'entre en jeu qu'au lot 4.

Un récapitulatif complet des routes figure en annexe.

3. Le contrat de qualité

Le responsable technique du réseau a posé des exigences non négociables, valables **dès le premier lot**. Elles ne rapportent pas de points : leur absence en fait perdre.

3.1 Typage

- TypeScript en mode **strict**.
- Zéro **any**, explicite ou implicite. Tout **@ts-ignore** est justifié en commentaire.
- Les réponses de l'API sont validées à l'exécution avec zod ou équivalent, pas seulement typées. Un type TypeScript ne protège de rien face à une API qui renvoie autre chose que prévu.

3.2 Architecture en couches

app/ réseau	écrans et routing (expo-router) : ni logique métier, ni appel
components/ features/ hooks/	interface pure, sans dépendance à l'API ni au store découpage par domaine (books, notes, auth, sync) logique réutilisable
services/ domain/ theme/	réseau, stockage, plateforme : seul endroit qui connaît l'API types et règles métier, sans dépendance technique tokens de design

Règle vérifiée en revue de code : aucun `fetch` et aucune URL en dur dans un fichier de `app/` ou de `components/`.

3.3 Gestion des erreurs

- Un type d'erreur applicatif discriminé : `ErreurReseau`, `ErreurValidation`, `ErreurConflit`, `ErreurAuth`.
- Aucun `catch` silencieux, aucun `console.log` résiduel.
- Un `ErrorBoundary` global, qui affiche un écran exploitable plutôt qu'un écran blanc.
- Toute action qui échoue produit un retour visible et une possibilité de réessayer.
- Les erreurs 422, qui signalent une validation par champ, et 503, qui signalent une indisponibilité passagère, ne se traitent pas de la même façon. L'API les distingue, votre interface aussi.

3.4 Tests

Avec `npm test` fonctionnel :

- les fonctions de domaine pures — tri, calculs, résolution de conflits — sont testées ;
- au moins trois composants testés avec `Testing Library` ;
- au moins un hook de données testé avec une API simulée (`MSW` ou mock de `fetch`) ;
- couverture indicative visée : 40 % minimum sur `domain/` et `services/`.

3.5 Git

- Une branche par fonctionnalité, des commits atomiques, des messages conventionnels (`feat`, `fix`, `refactor`, `test`, `docs`).
- Au moins une pull request par membre, relue par écrit par un coéquipier.
- La branche principale reste lançable à tout moment. Un dépôt à commit unique est éliminatoire sur ce critère.

3.6 Documentation

- `README.md` : un développeur extérieur lance le projet en moins de cinq minutes.
- `docs/ADR/` : au minimum trois décisions d'architecture documentées, format au chapitre 7.
- `IA.md` : voir chapitre 6.

4. Ce que le comité de recette refusera

- Du code que vous ne savez pas expliquer.
 - Filtrer, trier ou paginer côté client ce que le serveur sait faire.
 - Un fichier de plus de 250 lignes.
 - Une clé, un jeton ou un mot de passe en dur dans le dépôt.
 - Des types `any`, des `@ts-ignore` non justifiés, des `console.log` résiduels.
 - Un projet recopié depuis un dépôt public. Le contrôle porte sur l'historique Git et sur votre capacité à expliquer le code en direct.
-

5. Les lots de livraison

Les lots sont cumulatifs et correspondent chacun à un niveau de note. Le contrat de qualité du chapitre 3 s'applique à tous : un projet couvrant les fonctionnalités du lot 4 sans ce socle ne dépassera pas 13/20, tandis qu'un lot 3 impeccablement conçu, testé et documenté vaut mieux qu'un lot 4 bancal.

Lot 1 — Le cahier de lecture · 10/20

« Je veux retrouver sur un écran ce que j'avais dans mon cahier : la liste de nos ouvrages, et la possibilité d'en ajouter, d'en corriger, d'en supprimer. »

Fonctionnalités

- Liste des ouvrages du fonds, fiche détaillée, ajout, modification, suppression.
- Statut lu / non lu par l'équipe.
- Navigation entre trois écrans au minimum : liste, fiche, formulaire d'ajout et d'édition.
- Un ouvrage est défini par : titre, auteur, éditeur, année de publication, statut de lecture.

Exigences techniques

- Toute la couche réseau dans `services/api/`, avec un client HTTP unique centralisant l'URL de base, les en-têtes, les délais d'expiration et le traitement des erreurs.
 - Gestion de l'état serveur avec TanStack Query, ou justification écrite d'un choix alternatif : clés de cache structurées, invalidation après mutation.
 - Formulaire avec react-hook-form et zod : validation typée, message par champ, soumission désactivée pendant l'envoi, et remontée des erreurs 422 de l'API sur les bons champs.
 - Les quatre états sur chaque écran de données : chargement (squelette, pas un spinner plein écran), erreur avec réessai, vide contextualisé, succès.
 - Suppression avec confirmation, et possibilité d'annuler pendant cinq secondes — un libraire qui supprime la mauvaise fiche doit pouvoir se rattraper.
 - Pagination consommée correctement dès ce lot : l'écran ne charge pas les 500 ouvrages d'un coup.
-

Lot 2 — Les notes de lecture et les coups de cœur · 13/20

« Le cahier, ce n'est pas la liste des livres. C'est ce qu'on en dit. »

Fonctionnalités

- **Notes de lecture** rattachées à un ouvrage : affichage horodaté sur la fiche, ajout, suppression. C'est le cœur métier de l'application.
- **Coups de cœur** : un ouvrage peut être marqué favori, avec une icône cœur et une bascule instantanée.
- **Recherche et filtres côté serveur** : par titre et par auteur, filtres lus / non lus / coups de cœur, tri par titre, auteur, année ou note.
- Défilement infini ou pagination explicite, avec un indicateur de chargement de la page suivante distinct du chargement initial.
- Interface cohérente : thème centralisé, aucune couleur en dur, messages d'état soignés — aucun ouvrage, aucune note, ajout réussi.

Exigences techniques

- **Mises à jour optimistes** sur le coup de cœur et sur le statut de lecture, avec **retour arrière** si le serveur refuse. À démontrer en mode dégradé : un libraire qui clique sur le cœur doit voir l'effet immédiatement, et voir l'annulation si l'écriture échoue.
- Recherche avec anti-rebond de 300 ms et annulation de la requête précédente.
- Aucun rendu superflu : la frappe dans la barre de recherche ne doit pas re-rendre toute la liste. À prouver avec le profileur React DevTools.
- Accessibilité : rôle, libellé et état sur tous les éléments interactifs, zones tactiles d'au moins 44 points, navigation au clavier fonctionnelle — les postes de caisse n'ont pas tous une souris confortable.

Lot 3 — La fiche enrichie et le confort d'usage · 16/20

« Quand un client demande une édition précise, il faut que je puisse répondre sans quitter l'application. »

Fonctionnalités

- **Note interne de 0 à 5**, avec interaction par étoiles.
- **Couvertures** : affichage dans la liste et sur la fiche. L'API sert une couverture générée pour la plupart des ouvrages, mais pas pour tous, et le champ peut contenir un chemin relatif comme une URL externe — votre client doit traiter les trois cas sans jamais afficher une image cassée.
- **Remplacement d'une couverture** par le libraire : sélection d'un fichier, encodage en base64, envoi à `POST /books/:id/cover`, et retour possible à la couverture d'origine.
- **Enrichissement bibliographique** via OpenLibrary, à l'ouverture d'une fiche : `openlibrary.org/search.json?title=<titre>`. Affichez le nombre d'éditions référencées et, au choix, la première année de publication ou une couverture de secours. Attention : une partie du fonds a été saisie à la va-vite par les libraires et ne correspond à aucun ouvrage référencé. « Zéro édition trouvée » est une réponse normale, pas une erreur — traitez-la comme telle.
- **Thème clair et sombre** : bascule manuelle, respect de la préférence système par défaut, application globale via Context, persistance du choix. Les boutiques ouvrent tôt et ferment tard.
- **Interface bilingue français / anglais**, avec bascule à chaud, formats de date et de nombre compris — deux boutiques du réseau sont en zone frontalière.

Exigences techniques

- Appels OpenLibrary avec cache, anti-rebond et délai d'expiration. **L'indisponibilité d'OpenLibrary ne doit jamais casser la fiche** : dégradation silencieuse obligatoire.
- Une seule fonction, dans `services/`, résout une valeur du champ `couverture` en URL affichable : chemin relatif préfixé par l'URL de base, URL absolue laissée intacte, valeur nulle remplacée par un repli. Cette logique n'a rien à faire dans un composant.
- L'envoi d'une image gère les refus de l'API : format non supporté, image trop lourde. Une photo de téléphone brute dépasse la limite : redimensionnez avant l'envoi.
- Performance sur les 500 ouvrages : liste fluide, lignes mémoïsées, images dimensionnées et mises en cache, FlashList ou FlatList optimisée.
- Un document `docs/PERFORMANCE.md` présentant une mesure avant et après optimisation, avec la méthode employée.
- Aucune couleur et aucune chaîne visible par l'utilisateur en dur dans les composants.

Lot 4 — Comptes, mode hors ligne et synchronisation · 18-19/20

« En réserve, il n'y a pas de réseau. Au salon du livre non plus. Et je ne veux pas que le saisonnier puisse modifier les fiches. »

C'est le cœur du sujet. Ce lot n'est pas « ajouter le hors ligne » : c'est de la cohérence de données entre plusieurs postes, sous authentification.

4.1 Comptes et rôles

- Écran de connexion, session persistée, déconnexion.
- **Rafraîchissement automatique du jeton.** Le jeton d'accès expire toutes les 120 secondes. Cette expiration ne doit jamais être visible par le libraire, ni provoquer une déconnexion.
- **Routes protégées** : redirection vers la connexion, puis retour à l'écran initialement demandé une fois connecté.
- **Rôles** : un compte **lecteur** ne voit aucune action d'écriture — masquée, et non simplement désactivée. Un refus 403 renvoyé par l'API produit un message compréhensible.
- Jetons stockés derrière une abstraction **services/stockageSecurise.ts** : expo-secure-store sur mobile, repli documenté sur navigateur. **Le jeton de rafraîchissement ne transite jamais par un état React et n'apparaît dans aucun journal.**
- **Un intercepteur unique** : injection du jeton, détection du 401, rafraîchissement, rejeu de la requête. Si dix requêtes reçoivent un 401 en même temps, **un seul rafraîchissement** doit partir : les autres attendent le résultat. Ce point est systématiquement vérifié en recette.

4.2 Mode hors ligne

- **Cache local persistant** : au démarrage, l'application affiche immédiatement les données connues, puis revalide en arrière-plan. Un libraire qui ouvre l'application en réserve voit son fonds.
- **File de mutations** : hors ligne, l'ajout, la modification, la suppression d'un ouvrage et la rédaction d'une note de lecture restent possibles. Les mutations sont **persistées** — elles survivent à un rechargement complet de la page —, horodatées, et rejouées **dans l'ordre** au retour du réseau via **POST /sync**.
- **Indicateur de synchronisation** visible en permanence : en ligne, hors ligne, N modifications en attente, conflit à traiter.
- **Aucune perte de saisie** : couper le réseau au milieu de la rédaction d'une note ne doit rien effacer.
- Cas limite à traiter explicitement : que se passe-t-il si le jeton expire pendant la synchronisation d'un lot de mutations ?

4.3 Résolution des conflits

Deux libraires modifient la même fiche. Lorsqu'une mutation revient en **conflit**, l'application applique une stratégie explicite, documentée dans un ADR. Trois stratégies sont acceptées :

1. **Le serveur gagne** — la modification locale est écartée, le libraire est prévenu et peut récupérer sa version.
2. **Le client gagne** — réécriture forcée après rechargement de la version serveur.
3. **Fusion assistée** — écran de comparaison champ par champ, le libraire arbitre. C'est la plus valorisée.

Ce qui est évalué n'est pas la stratégie choisie, mais le fait qu'elle soit **choisie consciemment, implémentée réellement, et compréhensible par un libraire.**

4.4 Tableau de bord du responsable réseau

- Consommation de **GET /stats**, avec au moins deux graphiques : répartition lus et non lus, et distribution des notes ou répartition par année.
- Le tableau de bord reste consultable hors ligne à partir du cache, avec la date de dernière mise à jour affichée.

4.5 Exigences techniques du lot

- La logique de synchronisation est **pure et testée** : la fonction qui décide du sort d'une mutation face à une réponse serveur n'a aucun effet de bord et est couverte par des tests unitaires, cas de conflit compris.
- **Idempotence** : une même mutation rejouée deux fois ne crée pas deux ouvrages.

4.6 Le scénario de recette

Ce scénario sera rejoué devant vous, sur votre poste, pendant la soutenance. Préparez-le.

1. Se connecter en tant que libraire titulaire, puis passer hors ligne.
2. Créer un ouvrage hors ligne.
3. Modifier un ouvrage existant hors ligne.
4. Pendant ce temps, le responsable produit modifie le même ouvrage côté serveur, en ligne de commande.
5. Attendre l'expiration du jeton d'accès, soit 120 secondes.
6. Rétablir le réseau.

Attendu : le jeton est rafraîchi silencieusement, l'ouvrage créé hors ligne l'est une seule fois, le conflit est détecté, la stratégie annoncée s'applique, le libraire comprend ce qui s'est passé, et **aucune donnée n'est perdue silencieusement**.

Lot 5 — Mise en production · 20/20

Réservé aux projets qui, en plus du lot 4, présentent :

- une intégration continue GitHub Actions : lint, typecheck et tests à chaque pull request, avec badge dans le README ;
- des tests de bout en bout sur au moins un parcours critique, avec Maestro ou Playwright ;
- trois ADR de qualité professionnelle ;
- une revue de code croisée : votre équipe a produit une revue écrite et argumentée du dépôt d'une autre équipe, et a traité les remarques reçues sur le sien ;
- une journalisation structurée et une remontée d'erreurs ;
- un build EAS ou un export web déployé, accessible par lien ;
- un README qu'un développeur extérieur suit sans poser une seule question.

6. La politique interne sur l'IA

Le réseau autorise l'usage des assistants de génération de code, à une condition : **vous restez responsable de ce que vous livrez**. Ce qui compte n'est pas votre capacité à générer du code, mais votre capacité à juger celui qu'on vous propose.

Autorisé	Interdit
Générer, refactorer, tester, documenter	Livrer du code que vous ne comprenez pas
Faire expliquer une erreur ou une API	Faire générer le projet d'un bloc
Faire critiquer votre propre architecture	Omettre le fichier IA.md

Le fichier IA.md

Une page, individuelle même en équipe. Choisissez une fonctionnalité non triviale que vous avez fait générer — l'intercepteur d'authentification, la file de mutations, la résolution des conflits — et documentez :

- le **prompt exact** utilisé ;
- **trois défauts réels** que vous avez relevés dans le résultat. On attend des défauts d'ingénierie : condition de concurrence, mutation non idempotente, effet non nettoyé, erreur avalée, secret exposé, API inexistante dans la version installée ;
- les **corrections** apportées et leur justification.

En comité de recette, il pourra vous être demandé d'expliquer une portion de code générée, ligne par ligne et sans notes. Ne livrez rien que vous ne sachiez défendre.

7. Les livrables

7.1 Structure du dépôt

```

/
├── app/ components/ features/ hooks/ services/ domain/ theme/
├── docs/
│   ├── ADR/
│   │   ├── 001-gestion-etat-serveur.md
│   │   ├── 002-strategie-hors-ligne.md
│   │   └── 003-resolution-conflits.md
│   ├── PERFORMANCE.md
│   └── ARCHITECTURE.md
├── __tests__/
├── IA.md
└── README.md

```

docs/ARCHITECTURE.md contient le schéma des couches et le parcours complet d'une modification, du clic jusqu'au serveur.

7.2 Format d'un ADR

```

# ADR 003 – Stratégie de résolution des conflits

## Statut
Accepté – 12/11/2026

## Contexte
Les mutations hors ligne peuvent entrer en conflit avec des écritures
concurrentes.
Contraintes : X, Y, Z.

## Options envisagées
1. Le serveur gagne : simple, mais perte silencieuse de la saisie du libraire.
2. Le client gagne : écrase le travail d'un collègue.

```

3. Fusion assistée : coût d'implémentation et complexité de l'interface.

Décision

Nous retenons l'option 3, limitée aux champs textuels, avec repli sur l'option 1 pour les champs booléens.

Conséquences

Positives : ... Négatives : ... À revoir si : ...

7.3 Vidéo de démonstration

Deux minutes maximum, enregistrement d'écran du scénario de recette du lot 4. La recette ne dure que cinq minutes : cette vidéo est votre filet de sécurité si la démonstration en direct échoue, et la trace de ce que le comité n'aura pas eu le temps de voir.

8. Le comité de recette — 5 minutes

Cinq minutes, chronométrées. C'est court : rien ne s'improvise.

Durée	Séquence
2 min	Démonstration du scénario de recette, en direct, mode dégradé activé
1 min	Revue de code : vous ouvrez la couche de synchronisation et l'expliquez
2 min	Questions

Trois conseils. Répétez au moins une fois, chronomètre en main. Ne commencez pas par l'écran de connexion ni par une visite guidée des écrans : allez directement au scénario de recette, c'est lui qui a de la valeur. Ayez votre vidéo ouverte dans un onglet, prête à être lancée si la démonstration en direct se bloque.

Questions auxquelles vous devez savoir répondre

Deux d'entre elles vous seront posées, choisies au hasard, individuellement.

Architecture. Tracez le chemin complet d'une modification de fiche, du clic jusqu'au serveur. Où placeriez-vous une nouvelle entité « collections thématiques » ? Qu'est-ce qui, dans votre code, empêche un composant d'appeler directement l'API ?

Concurrence et réseau. Deux requêtes reçoivent un 401 en même temps : que se passe-t-il exactement ? Un libraire modifie un ouvrage puis le supprime hors ligne : que contient votre file de mutations ? Que se passe-t-il si la réponse d'une requête annulée finit par arriver ? Votre synchronisation est-elle idempotente, et comment le prouvez-vous ?

Données. Comment détectez-vous un conflit, et pourquoi ce mécanisme plutôt qu'un autre ? Que vaut `updatedAt` si l'horloge du poste de caisse est décalée de deux heures ? Que se passe-t-il si le stockage local est plein ou corrompu ?

Qualité. Quel test échouerait si je supprimais cette ligne ? Quelle est la partie du code dont vous êtes le moins fier, et pourquoi ? Quelle dette technique avez-vous acceptée sciemment ?

Usage de l'IA. Montrez-moi une ligne générée par une IA et expliquez-la. Quel défaut n'auriez-vous pas su repérer il y a trois mois ?

9. Planning conseillé

Demi-journée	Attendu
J1 matin	Dépôt initialisé, arborescence en couches, API lancée, ADR 001 rédigé, premier écran
J1 après-midi	Lot 1 complet, tests du domaine en place
J2 matin	Lot 2 : notes de lecture, coups de cœur, recherche serveur
J2 après-midi	Lot 3 : fiche enrichie, performance, thème
J3 matin	Lot 4 : comptes et intercepteur, puis file de mutations et conflits
J3 après-midi	Mode dégradé, ADR, IA.md, vidéo, gel du code à 14h, répétition de la démonstration

Trois jours, c'est court. Peu d'équipes atteindront le lot 4 en entier, et c'est normal : mieux vaut un lot 3 solide, testé et documenté qu'un lot 4 qui s'effondre à la première coupure réseau. Choisissez votre cible dès J1 et annoncez-la.

Le lot 4 n'est pas une couche à ajouter à la fin. Si votre couche **services/** n'est pas conçue dès le lot 1 pour accueillir une file de mutations et un intercepteur, vous réécrirez tout. **Lisez ce sujet en entier avant d'écrire la première ligne de code.**

Annexe — Récapitulatif de l'API

Le **README.md** du dossier **api-books-v2/** fait référence. Ce récapitulatif est un aide-mémoire.

Démarrage

```
npm install
npm run seed      # génère les 500 ouvrages du fonds et les comptes
npm start         # http://localhost:3000
```

Modèle de données

```
Ouvrage {
  id: string          titre: string      auteur: string
  editeur: string     annee: number      lu: boolean
  favori: boolean     note: number | null  couverture: string | null // URL
  ou null
  createdAt: string   updatedAt: string  version: number
}

Note {
  id: string          livreId: string   contenu: string   createdAt:
  string
}
```

Routes

Méthode et route	Effet
GET /health	État du serveur, mode dégradé, authentification
GET /books	Liste paginée et filtrée

Méthode et route	Effet
GET /books/:id	Fiche d'un ouvrage, en-tête ETag
POST /books	Création
PUT /books/:id	Remplacement complet, en-tête If-Match
PATCH /books/:id	Modification partielle
DELETE /books/:id	Suppression
GET /books/:id/notes	Notes de lecture d'un ouvrage
POST /books/:id/notes	Ajout d'une note
DELETE /books/:id/notes/:noteId	Suppression d'une note
GET /covers/:id.svg	Couverture générée, déterministe, jamais en erreur
POST /books/:id/cover	Envoi d'une image en base64
DELETE /books/:id/cover	Retrait de l'image envoyée
GET /stats	Statistiques du fonds
POST /sync	Synchronisation par lot
POST /auth/login	Connexion
POST /auth/refresh	Rafraîchissement du jeton
GET /me	Profil et rôle courants

Paramètres de GET /books

Paramètre	Valeurs	Défaut
page	entier positif	1
limit	1 à 100, plafonné	20
q	recherche titre et auteur	—
status	lu, nonlu	—
favori	true, false	—
sort	titre, auteur, annee, note, updatedAt	titre
order	asc, desc	asc

Les trois formes du champ couverture

Valeur	Signification
<code>/covers/<id>.svg</code>	Couverture générée par l'API, à préfixer par votre URL de base
<code>/media/<id>.png</code>	Image envoyée par un libraire, à préfixer également
<code>null</code>	Aucune couverture : affichez un repli, par exemple <code>/covers/<id>.svg</code>

Valeur	Signification
https://...	URL externe, à utiliser telle quelle

La route `/covers/:id.svg` répond pour n'importe quel identifiant, y compris un ouvrage créé hors ligne et pas encore synchronisé. Aucune fiche n'a donc de raison d'afficher un cadre vide.

Codes de réponse

Code	Signification	Ce que votre application doit faire
200 / 201 / 204	Succès	—
400	JSON illisible	Corriger la requête
401	Jeton absent, expiré ou invalide	Rafraîchir puis rejouer, sans déconnecter
403	Rôle insuffisant	Message clair, action masquée en amont
404	Ressource inconnue	État vide contextualisé
409	Conflit de version	Appliquer votre stratégie de résolution
413	Image trop lourde, lot de synchronisation trop grand	Redimensionner, ou découper le lot
415	Format d'image non supporté	Message clair : png, jpeg ou webp
422	Validation métier, champ par champ	Afficher l'erreur sous le bon champ
503	Service indisponible, mode dégradé	Réessayer avec temporisation, ne rien perdre

Comptes de test

Identifiant	Mot de passe	Rôle
editeur@booklist.fr	editeur123	Libraire titulaire, écriture

Identifiant	Mot de passe	Rôle
lecteur@booklist.fr	lecteur123	Libraire saisonnier, lecture seule